

Introdução à Computação lendo e escrevendo em arquivos

Alexandre Rademaker

scanf I

`scanf` recebe uma string template que contém designadores de tipos como `%i`, então a entrada padrão (STDIN) é lida até que o padrão seja reconhecido.

Par cada designador mencionado no template, um parâmetro adicional deve ser passado, um endereço para um tipo compatível com o designador.

`scanf` deverá efetivamente modificar o endereço indicado, preenchendo o valor lido, ou seja precisamos passar uma 'referência'.

scanf II

scanf1.c

```
1 #include <stdio.h>
2
3 int main(void) {
4     char *s = NULL;
5     printf("s: ");
6     scanf("%s", s);
7     printf("s: %s\n", s);
8 }
```

Veja o **bus error** ! Não alocamos memória para armazenar o valor lido. Mas como fazer isso? Qual tamanho devemos alocar?

scanf III

scanf2.c

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 // segmentation fault
5
6 int main(int argc, char *argv[]) {
7     char *s = malloc(10);
8     //char s[10];
9     printf("s: ");
10    scanf("%s", s);
11    //fgets(s, 10, stdin);
12    printf("s: %s\n", s);
13    free(s);
14 }
```

...ou seja, usar `scanf` para ler strings pode ser perigoso!

scanf IV

scanf3.c

```
1 #include <stdio.h>
2
3 int main(int argc, char *argv[]) {
4     // char *s = malloc(10);
5     char s[10];
6     printf("s: ");
7     scanf("%9s", s);
8     printf("s: %s\n", s);
9 }
```

e ainda existem `fgets` e `gets`. No MacOS ou Linux, use as **man** pages com `man gets`.

lendo e escrevendo em arquivos I

Todas as principais funções para ler e escrever em arquivos ficam na biblioteca `stdio.h`.

As funções mais comuns são: `fopen`, `fclose`, `fgetc`, `fputc`, `fread`, `fwrite`, `fscanf`, `fprintf`.

lendo e escrevendo em arquivos II

Veja arquivo phonebook.c

A função `fopen`, o tipo `FILE` e `fprintf` é a versão de `printf` que escreve em um arquivo.

lendo e escrevendo em arquivos III

antes

phonebook.csv

```
1 name,number
```

depois

phonebook.csv

```
1 name,number
2 Alexandre Rademaker,+55 21 12345
```


lendo e escrevendo em arquivos IV

Podemos ler arquivos jpeg

jpeg.c

```
1 #include <stdint.h>
2 #include <stdio.h>
3
4 typedef uint8_t BYTE;
5
6 int main(int argc, char *argv[]) {
7     if (argc != 2) return 1;
8
9     FILE *file = fopen(argv[1], "r");
10    if (!file) return 1;
11
12    // read first three bytes
13    BYTE bytes[3];
14    fread(bytes, sizeof(BYTE), 3, file);
15
16    if (bytes[0] == 0xff && bytes[1] == 0xd8 && bytes[2] == 0xff)
17        printf("Yes, possibly\n");
18    else
19        printf("No\n");
20
21    fclose(file);
22 }
```

lendo e escrevendo em arquivos V

O formato **BMP** é outro formato popular para armazenar imagens.

filter

```
1 #include "helpers.h"
2
3 void filter(int height, int width, RGBTRIPLE image[height][width])
4 {
5     // Loop over all pixels
6     for (int i = 0; i < height; i++)
7     {
8         for (int j = 0; j < width; j++)
9         {
10             image[i][j].rgbtBlue = 0x00;
11             image[i][j].rgbtGreen = 0x00;
12         }
13     }
14 }
```

A função acima percorre todos os pixels em um array bidimensional e define os valores de azul e verde como 0.